

Exploration B Notes

Vectors and Factors

At its core, R is an *objected*-oriented computational and programming environment. This means everything in R is an object with different properties. In this Exploration we will learn more about *vectors*, which are the core fundamental objects used in R, and will learn about a special type of vector called a *factor*.

Before getting too far into details about vectors, let's briefly discuss *data types*.

Data Types

R can represent different types of data. The types include `double`, `integer`, `complex`, `logical`, `character`, and `raw`. These are the basic fundamental objects we can use in R and are referred to as the `atomic` values. In this class, we will mainly use the `double`, `character`, and `logical` data types:

- **double:** Real numbers (numbers which may contain decimal values). Doubles can use decimal (12.34) or scientific form (.1234e2). When using numbers this is the default type used.
- **logical:** TRUE or FALSE values. You can create a vector that contains TRUE or FALSE directly, or you can also create a logical value without explicitly writing TRUE or FALSE but instead writing something that evaluates to TRUE or FALSE. For example: `3 > 2` is equivalent to TRUE while `3 == 2` is equivalent to FALSE.
- **character:** Words or text, usually referred to as *strings*. A *string* is a sequence of characters that must be surrounded by double quotes or single quotes, i.e. "Hello" and 'Goodbye'.

Vectors

A vector in R is a 1-dimensional object. That is, it contains a sequence of *elements* in a particular order. As we saw in Exploration A, to create an atomic vector we use the concatenate function `c()`, where each element of the vector is separated by a comma. This function will always create an atomic vector.

An atomic vector only contains elements of the same data type. If the function `c()` is given a mix of elements then it will convert these elements to be all of the same type according to a hierarchy of rules. The only exception to this is for `NA` values. The object `NA` is used to indicate missingness, or the lack of a value. The value `NA` can occur anywhere for all types of vectors.

Below are two examples of vector creation. The first is a vector with three elements; the first element is a 2, the second is a 6, and third is a 5. The second is a character vector that has just a single element, the string "a string of text". You'll notice that when creating a vector with a single element, you don't need to use the concatenate `c()` function.

```
# vector with three elements
v <- c(2,6,5)

# vector with one element
w <- "a string of text"
```

Let's review some of the features of vectors.

Combining Vectors

The concatenate function `c()` can be used to create vectors, but can also be used to combine vectors.

```
# creating two vectors
v1 <- c(1,2,3)
v2 <- c(4,5,6)

# combining the two vectors
v3 <- c(v1, v2)
v3
```

```
[1] 1 2 3 4 5 6
```

Vectorized Operations

As we saw briefly in Exploration A, vectors can be used in arithmetic computations. For example, below you can see an example of adding and multiplying two vectors. If you run the code block, you should see that it performs the addition and multiplication element by element.

```
# creating two vectors
v1 <- c(2, 4, 6)
v2 <- c(3, 5, 7)

# adding two vectors
v1 + v2
```

```
[1] 5 9 13
```

```
# multiplying two vectors
v1 * v2
```

```
[1] 6 20 42
```

You can also change every element of a vector by some constant number via the various arithmetic operations built into R. Below you can see examples of how to multiply every element in a vector by 5, divide every element by 3, and finally add 2 to every element and then multiply the resulting vector by 7.

```
# multiply every element by 5
5*v1
```

```
[1] 10 20 30
```

```
# divide every element by 3
v1/3
```

```
[1] 0.6666667 1.3333333 2.0000000
```

```
# add 2 then multiply by 7
(v1+2)*7
```

```
[1] 28 42 56
```

Accessing Elements of a Vector

Given a vector, you can perform operations with all of the elements in the vector. But you can also access individual elements of a vector by using an *index* in square brackets. An *index* is the location of an element in a vector. For example, the vector `v_dbl<- c(10, 11, 12)` has three elements; the number 10 has index is 1, the number 11 index is 2, and the number 12 has index 3. It's worth emphasizing that the first element is accessed using the index 1, not 0 - this is different than many other programming languages, where the first element is accessed using 0 (zero-based indexing).

There are a number of other indexing techniques that may be helpful:

- A negative index shows every element except the indicated index. So `v_dbl[-1]` is the vector `v_dbl` without the first element.
- The concatenate function can be used to obtain two or more elements of a vector in any desired order. Here `v_dbl[c(3,2)]` returns the third and second elements (in that order) of the vector `v_dbl`.

Some examples of the above techniques can be seen below.

```
# creating a vector
v_dbl <- c(10, 11, 12)

# Only get the third element
v_dbl[3]
```

```
[1] 12
```

```
# Get all elements except the first one
v_dbl[-1]
```

```
[1] 11 12
```

```
# Get the third and second element (in that order)
v_dbl[c(3,2)]
```

```
[1] 12 11
```

Named Vectors

Elements in a vector can also be given names. One can use the names instead of numeric indices to access elements. When you give the values in a vector names, this produces a *named vector*. The two techniques we will use to add names to vectors are (1) adding names when creating the vector, or (2) adding names to a vector after it has been created using the `names()` function.

To create a named vector using the first technique we use the `=` symbol where **the name is on the left of the equal sign, and the value being named is on the right**.

```
# Creating a named vector of ages
ages <- c(Anthony = 31, Maggie = 30, Shea = 2, Jack = 1)
ages
```

```
Anthony  Maggie    Shea   Jack
      31      30      2     1
```

Notice that when you display a named vector, **the names are displayed above the values in the vector**. You can then use the names to access elements from the vector. Notice that using this technique, you must use the `[]` brackets and `"` quotes around the name.

```
# obtaining Maggie's age
ages["Maggie"]
```

```
Maggie
      30
```

Note that you can still use numbers to index the vector if you want. For example;

```
ages[2]
```

```
Maggie
      30
```

The second technique for assigning names allows us to turn an already created vector into a named vector using the `names()` function.

```
# create an unnamed vector
president <- c("Abraham", "Lincoln")

# add names using names()
names(president) <- c("first", "last")

# displaying the named vector
president
```

```
      first      last
"Abraham" "Lincoln"
```

You can also use the `names()` function to see the list of names for a named vector.

```
# To see the names of a vector and not the elements
names(president)
```

```
[1] "first" "last"
```

If you want to remove the names of a named vector, redefine the vector you wish to remove the names of with the `unname()` function.

```
# Remove names using unname()
president <- unname(president)
president
```

```
[1] "Abraham" "Lincoln"
```

Length

To see how many elements are in a vector (named or otherwise), we can use the `length()` function.

```
length(president)
```

```
[1] 2
```

Factors

Factor objects are a special type of vector. As with vectors, they are a 1-dimensional sequence of elements, but factors are primarily used for categorical data, and are technically a special form of an integer type vector.

In real-world problems, you often encounter data that can be classified in categories. For example, suppose a survey was conducted on a group of seven individuals, who were asked to identify their hair color. We could create a factor to store their hair colors. The first step is to create a vector. **This is not yet a factor.**

```
# Creating a character vector of hair colors
hair <- c("Blonde", "Black", "Black", "Red", "Blonde", "Brown", "Black")
```

To turn this vector into a factor, you must use either the `factor()` or the `as.factor()` functions.

```
# Creating a factor using the hair vector
hair_fac <- factor(hair)
hair_fac
```

```
[1] Blonde Black  Black  Red    Blonde Brown  Black
Levels: Black Blonde Brown Red
```

```
# Alternative method to create the same factor
hair_fac <- as.factor(hair)
hair_fac
```

```
[1] Blonde Black  Black  Red    Blonde Brown  Black
Levels: Black Blonde Brown Red
```

Levels

You may have noticed in the previous code block that when we displayed the factor, it also displayed something called *levels*. Levels are one of the special properties of a factor object. For a given factor, R keeps track of all the distinct possible values in the factor, and each value is called a *level* of the associated factor. The `levels()` function is another way to show all the levels of a factor.

```
levels(hair_fac)
```

```
[1] "Black" "Blonde" "Brown" "Red"
```

If your factor contains only a subset of all the possible levels, then R will have an incomplete picture of the possible levels. Consider the following example of a vector consisting of directions. Notice that the obvious related value “South” is not included since there were no values of South in the factor.

```
directions <- c("North", "West", "North", "East", "North", "West", "East")
directions_fac <- factor(directions)
directions_fac
```

```
[1] North West North East North West East
Levels: East North West
```

However, in practice, it makes sense to have all the possible directions as levels of your factor. To add all the possible levels explicitly, you specify the `levels` argument of the function `factor()`, as shown in this example below.

```
# Make sure all possible categories are listed using the levels argument
directions_fac <- factor(directions,
                          levels = c("North", "East", "South", "West"))
directions_fac
```

```
[1] North West North East North West East
Levels: North East South West
```

If you ever want to remove any unused levels, you can simply use the `droplevels()` function.

```
# there are no ties in record
record = c("win", "loss", "loss", "win", "loss", "win")

f <- factor(record,
            levels = c("loss", "tie", "win"))

# this will remove "tie" from the levels since record doesn't contain any "tie"
droplevels(f)
```

```
[1] win loss loss win loss win
Levels: loss win
```

Combining Factors

Just as with vectors, the concatenate function `c()` can be used to combine factors.

```
f1 <- factor(c(1,2,2,2,3))
f2 <- factor(c(3,2,2,4))
f3 <- c(f1, f2)
f3
```

```
[1] 1 2 2 2 3 3 2 2 4
Levels: 1 2 3 4
```

Length

As with a vector, we use the `length()` function to see how many elements are in a factor.

```
length(f)
```

```
[1] 6
```

Names

Like standard vectors, we can also make named factors. We can either start with a named vector, or add names to a factor using the `names()` function just like we did in the standard vector method.

```
# creating a named vector
record_vec <- c(sally = "win", tom = "win", ed = "lost", jane = "tie")
record <- factor(record_vec)
record
```

```
sally  tom    ed  jane
   win   win lost  tie
Levels: lost tie win
```

Accessing Elements

We can also access elements of a factor object using the same standard techniques described for accessing elements in a vector Exploration B.

```
# Obtain the first element
record[1]
```

```
sally
  win
Levels: lost tie win
```

```
# Obtain the fourth and second elements
record[c(4, 2)]
```

```
jane tom
  tie win
Levels: lost tie win
```

```
# Obtain the second element using its name
record["tom"]
```

```
tom
win
Levels: lost tie win
```

The table() function

When dealing with categorical data, as we do with factors, a very useful function is the `table()` function. The table function tabulates observations. In the example below, when we apply the `table()` function to a factor, it produces a *named vector* where the names are the levels of the factor and the values are the number of instances of that level in the factor. See the example below.

```
table(hair)
```

```
hair
Black Blonde Brown Red
      3      2      1      1
```

Because the `table()` function produces a named vector, it has all of the features associated to a named vector, such as accessing the values in the table using numeric or name indices.

```
# accessing the first element of the table
table(hair)[1]
```

```
Black
      3
```

```
# accessing the value with name "Red"
table(hair)["Red"]
```

```
Red
    1
```

The `table()` function can also be used to tabulate two-way frequency tables. The following code creates a two-way frequency table showing how many people with each hair color own or don't own pets. Notice that two factors of the same length were created such that the first indices of each correspond to each other (first person is blonde and owns a pet), the second indices corresponds to each other (second person is black-haired and does not own a pet), and so on.

```
hair <- c("Blonde", "Black", "Black", "Red", "Blonde", "Brown", "Black")
own_pets <- c(TRUE, FALSE, TRUE, TRUE, FALSE, FALSE, FALSE)

hair <- factor(hair)
own_pets <- factor(own_pets)

# Two way table
table(hair, own_pets)
```

	own_pets	
hair	FALSE	TRUE
Black	2	1
Blonde	1	1
Brown	1	0
Red	0	1

Notice that the first vector you plug into `table()` becomes the rows and the second becomes the columns.

The %in% Operator

The %in% operator is useful when working with vectors and factors. It allows you to check if a certain value is in a vector or factor. It returns **TRUE** if the specified value is in the vector or factor and **FALSE** otherwise.

```
# returns TRUE  
"Red" %in% hair
```

```
[1] TRUE
```

```
# returns FALSE  
"Purple" %in% hair
```

```
[1] FALSE
```